Technical Roadmap

Version: 1.0

Purpose

This document describes the technical implementation roadmap for Trading Platform Pro and its primary product, the Trading Cockpit.
Unlike `Product_Roadmap.md`, this roadmap focuses on:
• technical implementation priorities
• capability delivery order
• architecture evolution
• engineering milestones
• operational safety
• quality gates
The technical roadmap follows concrete Trading Cockpit requirements.
Shared platform capabilities evolve when multiple implemented product capabilities require them.

Roadmap Principle

The Trading Cockpit is developed through vertical product capabilities.
Preferred evolution:
```text
Trading Cockpit Requirement
↓
Vertical Product Capability
↓
Application and Domain Behaviour
↓
Required Infrastructure
↓
Presentation Integration
```
Do not build generic platform infrastructure before a concrete product capability requires it.
Shared infrastructure shall emerge from demonstrated reuse.

Product-First Delivery

The roadmap does not separate development into:
```text
Build Generic Platform
↓
Build Trading Platform
↓
Build Trading Cockpit
```
Instead, each phase delivers or strengthens concrete Trading Cockpit capabilities.
A vertical capability may include:
```text
Domain
Application
Infrastructure
Presentation
Testing
Documentation
```

```
while preserving architecture dependency direction.
---
# Current Direction

The current engineering direction is:
• Trading Cockpit first
• vertical product slices
• explicit capability ownership
• deterministic state transitions
• explicit external side effects
• strong PAPER and LIVE separation
• broker acknowledgement semantics
• duplicate submission protection
• reconciliation-aware workflows
• controlled AsyncIO lifecycle
• synchronized Markdown documentation
• automated quality gates
Generic platform capabilities are not roadmap objectives by themselves.
---
# Phase 1 – Runtime and Safety Foundation

Status: **In Progress**
Objective:
Establish the technical foundation required for safe and deterministic Trading Cockpit operation.
Primary areas:
• configuration
• dependency registration
• runtime lifecycle
• AsyncIO integration
• structured logging
• monitoring
• persistence foundation
• controlled startup
• controlled shutdown
• PAPER and LIVE separation
• source quality gates
• automated testing foundation
• documentation generation
Required outcomes:
• runtime state is explicit
• startup order is deterministic
• shutdown order is deterministic
• long-running tasks have explicit ownership
• cancellation behaviour is defined
• timeout behaviour is explicit
• PAPER remains PAPER
• LIVE requires explicit selection
• no fallback activates LIVE
• Markdown remains documentation source of truth
• required CI quality gates are executable
The foundation shall support concrete Trading Cockpit capabilities.
It shall not become a generic platform-services program.
---
# Phase 1 Safety Milestones
```

Before trading-critical workflows are considered operationally mature:
• broker connection state is explicit
• market data connection state is explicit
• broker and market data lifecycles remain distinguishable
• external side effects are Application-owned
• financial data availability remains explicit
• persistence state is deterministic
• monitoring remains observational
• reconciliation observation is separated from repair
• automated tests do not require LIVE trading
Trading-critical infrastructure shall not rely solely on logs as authoritative business state.

Phase 2 – Market Monitoring

Objective:
Deliver the market observation capabilities required by the Trading Cockpit.
Primary capabilities:
• market data connectivity
• quote retrieval
• quote subscriptions
• source timestamps
• freshness state
• stale data handling
• unavailable data handling
• watchlists
• market overview
• scanner foundations
• market monitoring presentation
Required technical outcomes:
• provider models remain in Infrastructure
• market data ports expose project-owned contracts
• source timestamps are preserved
• current, stale, unavailable and disconnected states remain distinguishable
• subscriptions have explicit lifecycle ownership
• active subscriptions can be closed deterministically
• cached data does not silently appear as current provider data
Market monitoring shall not contain trading decision rules.

Phase 2 Validation

Required validation includes where implemented:
• quote translation tests
• unavailable data tests
• stale data tests
• connection state tests
• subscription activation tests
• subscription failure tests
• subscription closure tests
• runtime lifecycle tests
• controlled shutdown tests
PAPER or controlled provider validation may supplement deterministic tests.

Phase 3 – Decision Center

Objective:
Deliver explicit Trading Cockpit decision workflows.

Primary capabilities:
• Trading Candidates
• candidate review
• candidate lifecycle
• Trading Decisions
• decision rationale
• decision state
• decision presentation
• decision history
Required technical outcomes:
• Trading Candidate and Trading Decision remain separate Domain concepts
• candidate review is Application-owned
• decision workflows are explicit
• state transitions are deterministic
• rationale remains traceable
• presentation does not implement trading rules
• provider data does not leak directly into Domain decisions
A Trading Decision does not automatically imply order submission unless an explicit Application workflow owns that transition.

Phase 3 Validation

Required validation includes where implemented:
• candidate lifecycle tests
• decision lifecycle tests
• valid state transition tests
• invalid state transition tests
• Application workflow tests
• persistence tests
• presentation interaction tests
Decision tests shall use deterministic market and time inputs.

Phase 4 – Order Management

Objective:
Deliver safe and explicit order workflows.
Primary capabilities:
• order creation
• order validation
• order submission
• broker acknowledgement
• broker rejection
• partial fill processing
• fill processing
• order cancellation
• order history
• order presentation
Order Management is trading-critical.

Phase 4 Order Lifecycle

The order workflow shall preserve relevant lifecycle distinctions.
```text
Submission Requested
↓
Validated
```

↓
Transmitted
↓
Acknowledged or Rejected
↓
Partially Filled or Filled
```

Cancellation introduces separate explicit states where required.
A successful adapter method call does not automatically mean broker acknowledgement.
Broker acknowledgement requires broker-derived state.
---

# Phase 4 Duplicate Submission Protection

Before order submission is operationally mature, the workflow shall address:
• stable order identity
• stable submission identity
• repeated callbacks
• repeated broker messages
• timeout
• disconnect
• reconnect
• application restart
• recovery
Unknown external order state shall not automatically trigger resubmission.
Automatic order submission retry is prohibited unless explicitly defined and approved by the owning Application workflow.
---

# Phase 4 Validation

Required regression scenarios include where implemented:
• valid order submission
• invalid order submission
• local validation failure
• transmission
• broker acknowledgement
• broker rejection
• timeout
• disconnect
• reconnect
• duplicate submission prevention
• repeated acknowledgement
• partial fill
• repeated execution message
• fill
• cancellation request
• cancellation acknowledgement
• execution during cancellation
Trading workflow regression tests are mandatory for implemented business-critical order scenarios.
PAPER validation shall supplement, not replace, deterministic automated tests.
---

# Phase 5 – Portfolio and Reconciliation

Objective:
Deliver deterministic portfolio state and explicit reconciliation workflows.
Primary capabilities:
• positions
```

• position lifecycle
• portfolio state
• execution-derived position changes
• broker state comparison
• discrepancy detection
• discrepancy classification
• action-required state
• reconciliation presentation
Required technical outcomes:
• local business state remains explicit
• broker state remains externally derived state
• position state is not inferred from UI state
• reconciliation observation is separated from repair
• monitoring does not repair Domain state
• logging does not repair Domain state

Phase 5 Reconciliation

Reconciliation may identify:
• missing local order
• missing broker order
• order state mismatch
• position state mismatch
• quantity mismatch
The initial reconciliation workflow should:
```text
Load Local State
↓
Load Broker State
↓
Compare
↓
Classify Discrepancy
↓
Expose Action-Required State
```
Repair requires an explicit Application workflow.
Do not silently rewrite business history.

Phase 5 Validation

Required validation includes where implemented:
• position opening
• position update
• partial quantity changes
• position closing
• position persistence
• no-discrepancy reconciliation
• discrepancy detection
• discrepancy classification
• reconciliation failure
• repeated reconciliation invocation
If repair workflows are introduced, they require separate authorization and regression tests.

Phase 6 – Risk Management

Objective:
Deliver explicit risk capabilities based on implemented Trading Cockpit workflows.
Primary capabilities may include:
• order risk validation
• position risk
• portfolio exposure
• concentration
• trading limits
• risk alerts
• risk presentation
Risk Management shall evolve from concrete Trading Cockpit requirements.
Do not build a generic enterprise risk framework without demonstrated product need.

Phase 6 Risk Ownership

Risk rules belong to Domain or Application according to responsibility.
Broker adapters shall not own:
• portfolio risk decisions
• candidate acceptance rules
• order risk policy
Presentation shall not implement risk decisions.
Risk validation affecting order submission shall occur before the external order side effect.

Phase 6 Validation

Required validation includes where implemented:
• valid risk state
• invalid risk state
• order risk rejection
• limit boundary conditions
• unavailable financial data
• portfolio exposure calculations
• deterministic financial precision
Risk tests shall preserve the distinction between unavailable values and zero.

Phase 7 – Reporting and Analytics

Objective:
Deliver traceable reporting and analytics for implemented Trading Cockpit capabilities.
Primary capabilities may include:
• reporting
• performance analysis
• trading journal
• execution analysis
• risk analytics
• operational analytics
• statistics
AI-assisted insights may be evaluated when concrete product requirements and reliable source data exist.
AI functionality is not a mandatory platform milestone.

Phase 7 Data Principles

Reporting and analytics shall use explicit source semantics.
Reports shall distinguish where relevant:

• local business state
• broker-derived state
• market data state
• reconciled state
Do not silently invent missing financial values.
Derived analytics shall remain traceable to authoritative source data where practical.

Phase 7 Validation

Required validation includes where implemented:
• calculation tests
• financial precision tests
• unavailable data tests
• source-state tests
• report generation tests
• deterministic analytics tests
Analytics tests shall not depend on current market state.

Phase 8 – Shared Capabilities as Required

Objective:
Extract or strengthen shared capabilities only when multiple concrete product capabilities demonstrate the need.
Potential examples:
• shared scheduling
• shared resource lifecycle
• shared messaging
• shared transaction coordination
• shared import and export
• shared extension boundaries
These are not automatic roadmap commitments.
Each shared capability requires:
1. At least one concrete requirement.
2. Identified consumers.
3. Clear ownership.
4. Defined contract.
5. Migration analysis.
6. Test strategy.
Prefer extraction from demonstrated reuse over speculative platform construction.

Plugin and Extension Strategy

A Plugin System, Plugin SDK or general extension ecosystem is not a current roadmap milestone.
Such capabilities may be evaluated when:
• a concrete extension requirement exists
• at least one real extension consumer exists
• lifecycle ownership is defined
• security impact is understood
• compatibility requirements are explicit
Do not introduce plugin infrastructure solely for hypothetical future extensibility.

Public API Strategy

Public APIs are not a default roadmap milestone.
A public API requires:

• concrete external consumer
• explicit contract ownership
• versioning policy
• compatibility policy
• authentication and authorization requirements
• operational ownership
Internal project-owned ports are not automatically public APIs.

Cloud Strategy

Cloud readiness is not a generic roadmap objective.
Cloud-related architecture shall be introduced only when a concrete deployment requirement exists.
Do not redesign local runtime, persistence or integration architecture solely to claim cloud readiness.

Cross-Phase Engineering Activities

The following activities continue throughout relevant phases:
• focused refactoring
• documentation synchronization
• automated testing
• trading workflow regression protection
• architecture boundary validation
• source quality verification
• security review
• dependency maintenance
• technical debt reduction
Performance optimization is performed after measurement.
Do not optimize speculative bottlenecks.

Documentation Milestones

Documentation is part of each capability implementation.
When a capability changes documented:
• architecture
• behaviour
• API
• configuration
• runtime lifecycle
• workflow
• operational state
the affected Markdown documentation shall be updated before final verification.
When Markdown changes run:
```bash
python scripts/generate_docs.py
```
Documentation generation shall complete successfully before commit.

Testing Milestones

Testing depth follows operational risk.
Typical sequence:
```text
Unit Tests
↓
```

Integration Tests
↓
Trading Workflow Regression Tests
↓
Runtime and System Tests
↓
Explicit PAPER Validation
```

LIVE validation requires explicit approval.
LIVE validation is never part of normal CI.
---

# Architecture Evolution

Architecture shall evolve with implemented product capabilities.
Before introducing a significant architecture change:
1. Identify the concrete requirement.
2. Inspect the current implementation.
3. Review current architecture documentation.
4. Identify the owning capability.
5. Identify real consumers.
6. Evaluate alternatives.
7. Evaluate migration impact.
8. Evaluate trading safety where relevant.
9. Create or update an ADR where durable architecture rationale is required.
10. Synchronize current architecture documentation.
Do not introduce architecture solely for hypothetical future scalability.
---

# Phase Entry Criteria

Before starting a major capability phase verify:
• product requirement is defined
• capability ownership is identified
• architecture boundaries are understood
• required dependencies exist or are explicitly scoped
• persistence impact is evaluated
• runtime impact is evaluated
• testing strategy is defined
• documentation impact is identified
For trading-critical phases additionally verify:
• external side effects identified
• duplicate submission risk evaluated
• retry behaviour evaluated
• broker acknowledgement semantics evaluated
• reconciliation impact evaluated
• PAPER and LIVE impact evaluated
---

# Phase Completion Criteria

A phase or capability milestone is complete only when relevant outcomes are implemented and verified.
Verify:
• required capability behaviour exists
• architecture boundaries remain consistent
• state transitions are deterministic
• external side effects are explicit
• focused tests pass
```

• required regression tests pass
• source quality checks pass
• documentation is synchronized
• documentation generation passes
• operational impact is reviewed
For trading-critical capabilities additionally verify:
• duplicate submission protection
• broker acknowledgement semantics
• timeout behaviour
• disconnect behaviour
• reconnect behaviour
• reconciliation impact
• PAPER validation where required
Do not mark a capability complete while known business-critical behaviour remains unresolved.

Roadmap Governance

The technical roadmap shall:
• align with `Product_Roadmap.md`
• prioritize Trading Cockpit requirements
• preserve architecture dependency direction
• support vertical product capabilities
• avoid speculative platform infrastructure
• remain reviewable
• evolve after major product or architecture decisions
Roadmap phases express implementation priority.
They are not permission to implement every listed capability without a concrete requirement.

Roadmap Change Process

When roadmap direction changes:
1. Identify the product or technical trigger.
2. Review `Product_Roadmap.md`.
3. Review `Decision_Log.md`.
4. Review relevant ADRs.
5. Evaluate affected phases.
6. Update this roadmap.
7. Update dependent documentation.
8. Record a significant product or process decision where required.
9. Create an ADR where architecture-significant rationale is required.
Do not silently change roadmap direction through isolated implementation work.

Success Criteria

The technical roadmap is successful when:
• Trading Cockpit capabilities deliver concrete product value
• architecture boundaries remain consistent
• capability ownership remains clear
• external side effects remain explicit
• trading workflows remain deterministic
• duplicate submission risk is controlled
• broker acknowledgement semantics remain explicit
• PAPER and LIVE remain separated
• reconciliation remains explicit
• documentation stays synchronized
• automated regression protection grows with implemented risk

• technical debt remains controlled
Success is measured by safe and maintainable product capability delivery.
It is not measured by the number of generic platform services implemented.

Roadmap Review Checklist

Before starting or reprioritizing a major phase verify:
• concrete Trading Cockpit requirement exists
• Product Roadmap alignment confirmed
• Decision Log reviewed
• relevant ADRs reviewed
• capability ownership identified
• real consumers identified
• speculative infrastructure avoided
• architecture boundaries understood
• dependencies evaluated
• runtime impact evaluated
• persistence impact evaluated
• external side effects evaluated where relevant
• duplicate submission risk evaluated where relevant
• retry behaviour evaluated where relevant
• broker acknowledgement evaluated where relevant
• reconciliation impact evaluated where relevant
• PAPER and LIVE impact evaluated where relevant
• testing strategy defined
• documentation impact identified
• implementation priorities confirmed

Related Documents

• Product_Vision.md
• Product_Roadmap.md
• Project_Overview.md
• Architecture.md
• Domain_Model.md
• Infrastructure.md
• Technical_Specifications.md
• Decision_Log.md
• Runtime.md
• Configuration.md
• Testing_Strategy.md
• Development_Guidelines.md
• CI_CD.md
• AI_Agent_Guide.md
• AGENTS.md
• docs/adr/README.md